

车载视觉道路障碍物与未知 风险预警系统

系统架构设计技术报告

Driver Vision Road Obstacle and Unknown Risk Warning System
System Architecture Design Report

报告范围: 系统架构设计分析

Repository: yyh061129vic-debug/driver-vision-risk-warning-system

日期: 2026-08-10

目录

1	引言 Introduction	3
1.1	项目背景与动机	3
1.2	项目目标	3
1.3	当前开发状态	4
1.4	报告范围	4
2	系统架构 System Architecture	5
2.1	总体系统架构	5
2.2	感知层设计	7
2.3	可行驶区域分割模块	8
2.4	目标检测模块	10
2.5	空间风险分析	11
2.6	数据流与接口设计	12
2.7	架构设计决策	14
2.8	当前实现与未来扩展	15
3	讨论 Discussion	16
3.1	架构优势分析	16
3.2	当前架构局限	17
3.3	后续扩展方向	18

1. 引言 Introduction

1.1 项目背景与动机

城市道路环境中存在施工锥桶、临时护栏、纸箱、轮胎、木板、散落货物、坑洼以及事故占道物等多种风险源。传统封闭集识别模型仅能处理预先定义的类别集合，面对训练分布之外的未知障碍物时可能产生漏检，从而在自动驾驶辅助场景中留下安全隐患。本项目旨在建立一条「可行驶区域分割 - 已知障碍物检测 - 未知风险发现 - 空间冲突判断 - 风险预警」的车载视觉处理链路，通过语义分割与目标检测的互补感知，结合空间推理与距离估计，实现对已知和未知道路风险的实时预警。

1.2 项目目标

系统设计目标涵盖以下五个核心能力：

- 实时输出道路可行驶区域及其边界，为后续空间冲突判断提供像素级道路掩码。
- 检测施工锥桶、路障、遗撒物、坑洼和静止占道物等已知风险，输出边界框与类别。
- 发现训练类别之外、但真实占据道路空间的未知障碍物，以异常区域形式输出。
- 结合距离估计、自车速度和预计行驶走廊，输出 Normal / Warning / Danger 三级风险等级。
- 在夜间、雨雪、逆光和部分遮挡条件下保持可用，并在传感器严重退化时正确输出降级状态。

1.3 当前开发状态

截至报告撰写日期，项目已完成第一周启动清单中的六项任务：仓库与标准目录建立、候选数据调研与许可预审、本地开发环境基线记录、20张道路与异常样例可视化、可行驶区域分割 Demo 以及第一版实验方案冻结。V1 实验方案已固定数据划分、SegFormer 模型 revision、512x512 实际模型输入尺寸、随机种子和三项主要质量指标(Binary mIoU、Road IoU、Boundary F-score)。

需要特别说明的是，当前已跑通的核心模块为可行驶区域分割子系统(基于 SegFormer-B0 的推理流程)，而障碍物检测、未知异常发现、距离估计和风险判断仍属于规划中的后续模块。团队提交区中已包含 YOLOv11s-P2 目标检测训练代码和 SegFormer + YOLO 融合推理原型，但这些代码尚未集成到主工程链路中。

1.4 报告范围

本报告聚焦于系统架构设计分析，不包含方法论(Methodology)和实现细节(Implementation)章节。报告将围绕以下核心问题展开：系统为何采用这样的架构？各模块之间如何连接？数据

如何 流动？当前架构解决了什么问题？还存在哪些架构缺陷和后续扩展空间？ 报告的目标读者为系统架构评审人员和项目组成员，旨在提供对系统整体设计的清晰理解，而非对 SegFormer 或 YOLO 等模型的科普介绍。

2. 系统架构 System Architecture

2.1 总体系统架构

系统采用模块化的「感知-风险」分层架构，整体处理链路从图像采集出发，经过感知、场景理解、风险评估，最终输出风险预警。架构设计分为五个层次：输入层、感知层、场景理解层、风险评估 层和预警层。每一层职责独立，通过结构化接口向上层传递结果。

系统规划的目标架构覆盖从图像输入到风险预警的完整链路。然而，当前实际已跑通的仅为第一阶 段的感知子系统，即可行驶区域分割模块。目标架构与当前实现之间的差异是本报告架构分析的核 心要点之一。

目标架构涵盖图像与自车状态接入、质量检查、可行驶区域分割、已知障碍物检测、未知异常发现、 距离估计、行驶走廊预测、空间冲突与风险计算、状态与日志输出等完整环节。当前实现则集中在 可行驶区域分割 Demo 上：输入图像经过预处理后送入 SegFormer-B0 模型，输出语义分割结果，经过双线性上采样、softmax 和 argmax 后处理后生成二值道路掩码、边界和置信度图，最终通过可 视化叠加输出。

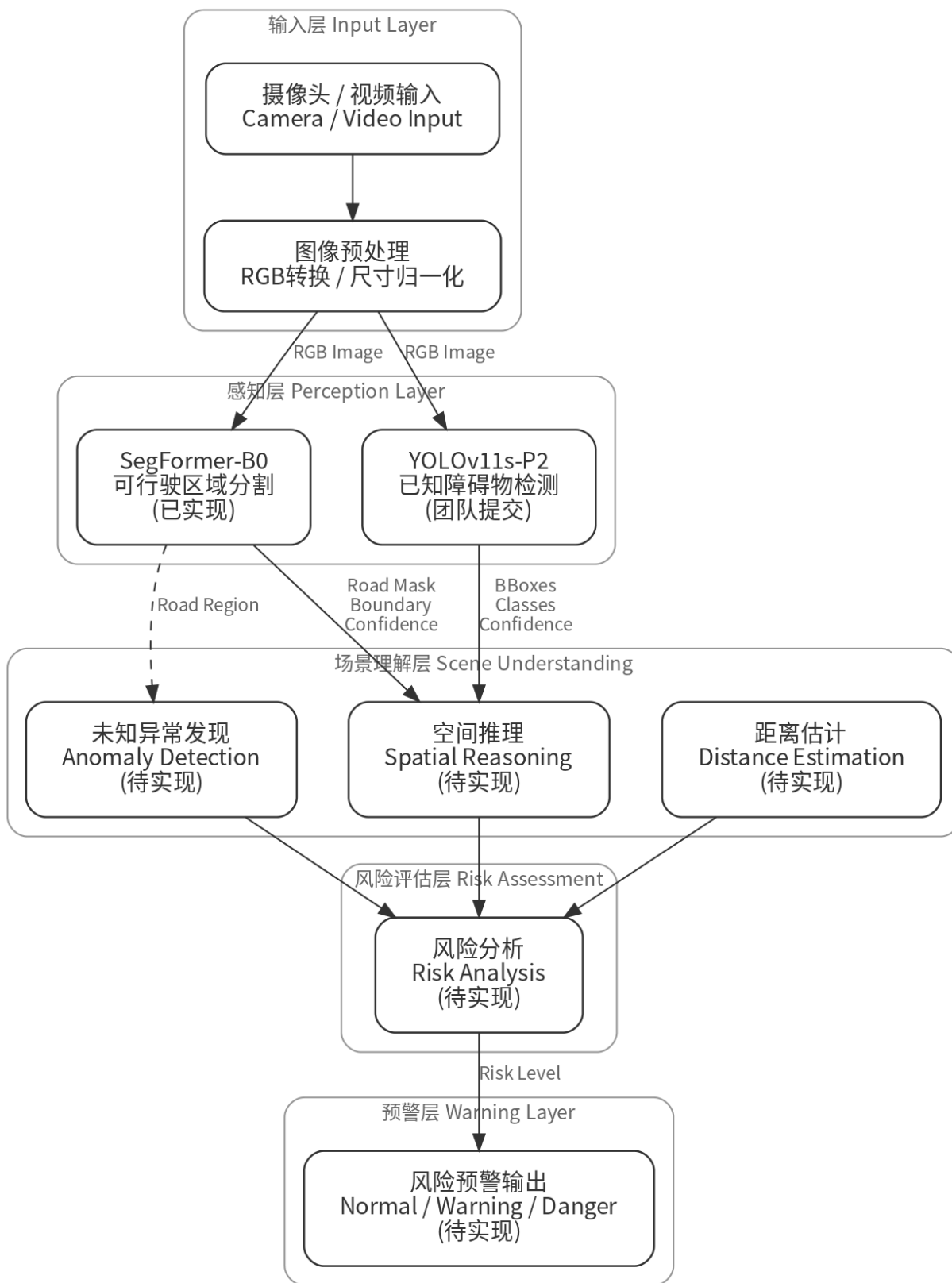


图 1: 系统总体架构 - 目标架构与当前实现状态对照

上图中，绿色标注的模块 (SegFormer 分割、图像预处理) 为已实现部分，黄色标注的模块 (YOLO 检测) 已有团队提交代码但尚未集成，橙色和粉色标注的模块 (空间推理、异常发现、距离估计、风险分析、预警输出) 仍处于规划阶段。这一分层设计确保了各模块可以独立开发和测试，通过结构化接口实现渐进式集成。

2.2 感知层设计

感知层是整个系统的核心，采用互补感知 (Complementary Perception) 设计策略。语义分割和目标检测并非重复工作，而是提供两种不同维度的场景信息：SegFormer 回答「车辆可行驶区域在哪里」(Where can the vehicle drive?)，提供像素级道路区域理解；YOLO 回答「场景中有什么物体」(What objects are present?)，提供目标级障碍物检测。两者的输出在后续场景理解层进行空间推理融合。

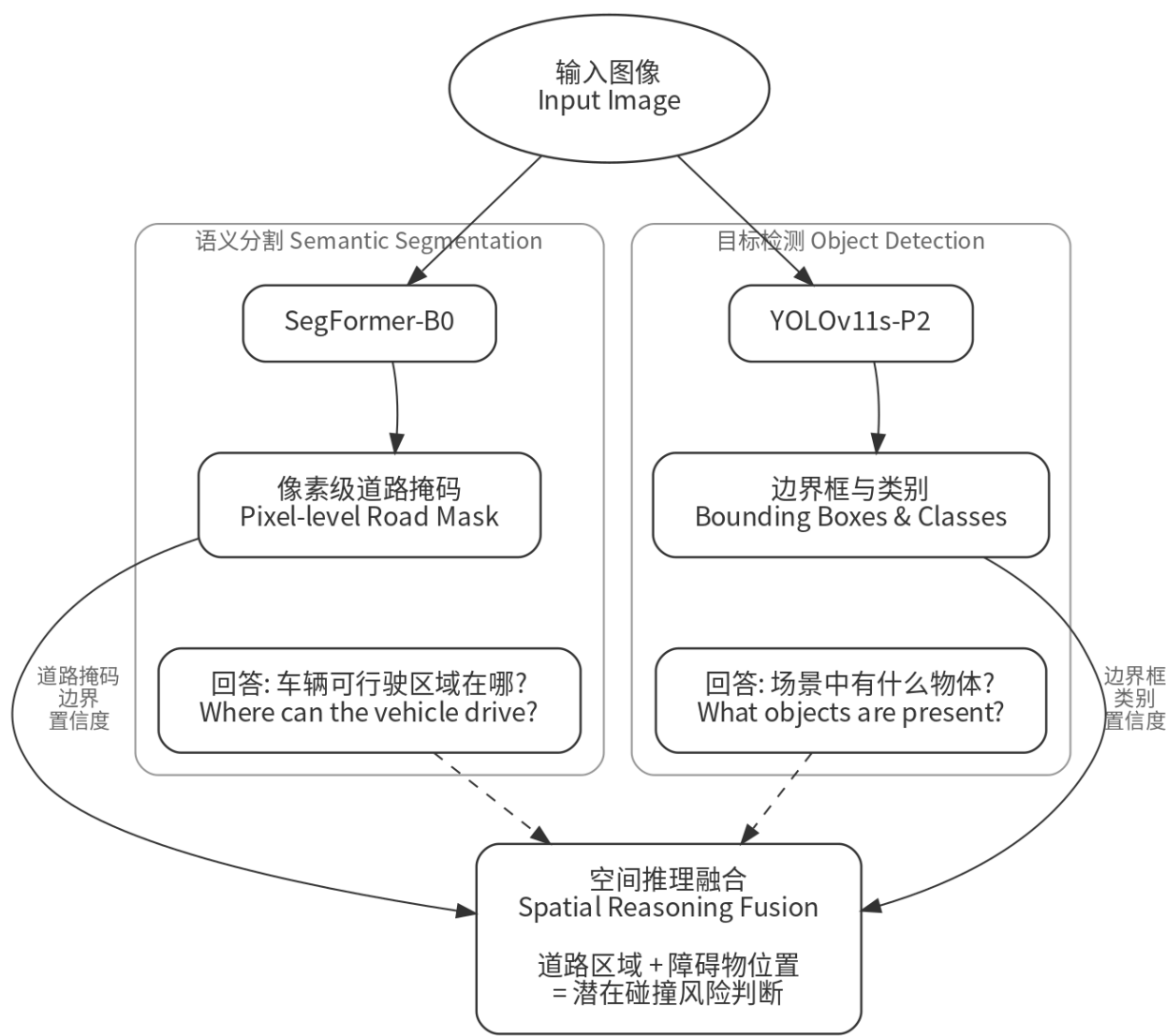


图 2: 互补感知设计 - SegFormer 语义分割与 YOLO 目标检测

SegFormer-B0 作为轻量级 Transformer 分割模型，从 Cityscapes 预训练权重加载，输出 19 类像素级语义分割结果，其中类别 0 (road) 被提取为可行驶区域。YOLOv11s-P2 模型则针对 15 类道路目标进行检测，包括 bus、car、truck、person、bike、cone、lost_tire、obstacle 等，覆盖已知障碍物的主要类别。两个模型以并行方式处理同一帧图像，各自独立输出，不构成计算依赖。

这种互补设计的关键优势在于：SegFormer 的像素级道路掩码可以精确定义车辆的可行行驶空间，而 YOLO 的边界框检测可以识别占据该空间的已知物体。当某物体的边界框与道路掩码区域存在空间交叠时，即触发潜在碰撞风险判断。此外，道路掩码中未被任何已知目标检测框覆盖的

异常区域，可进一步用于未知障碍物的发现。

2.3 可行驶区域分割模块

可行驶区域分割是当前已完整跑通的核心子系统。该模块基于 SegFormer-B0 模型，遵循「输入-预 处理-推理-后处理-输出」的标准流程，所有像素级输出均保持输入图像的原始分辨率。

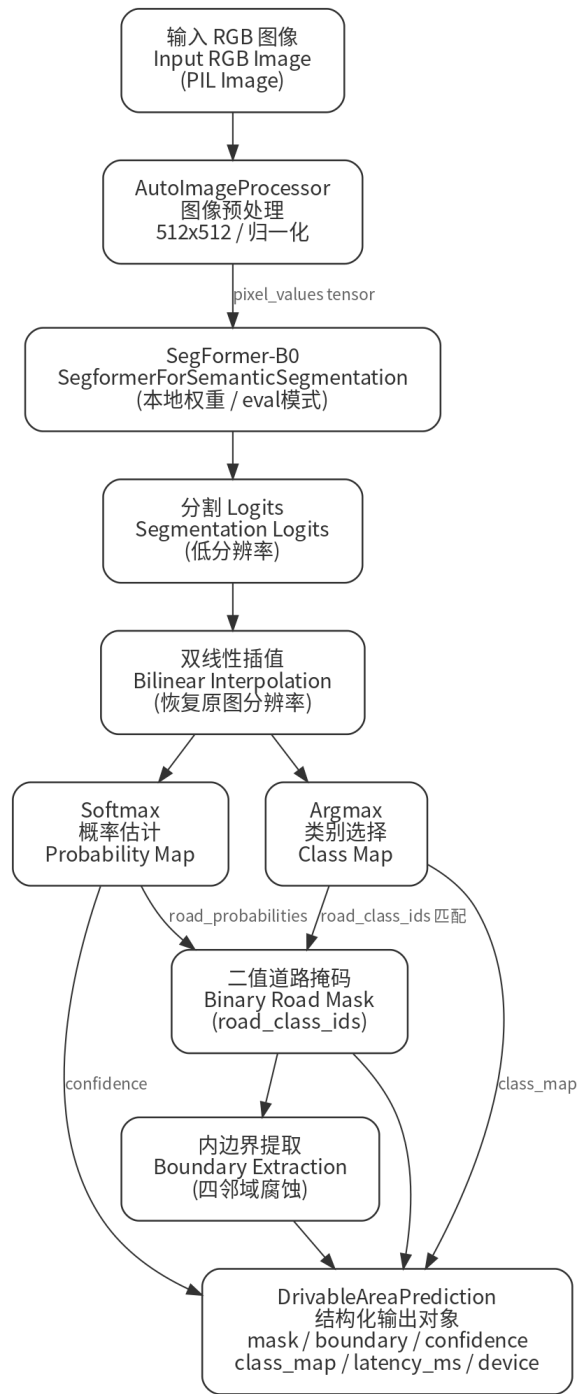


图 3: SegFormer-B0 可行驶区域分割推理流程

推理流程各步骤说明

1. 输入	接收 PIL Image 并转换为 RGB	convert('RGB')
-------	-----------------------	----------------

2. 预处理	AutoImageProcessor 将图像缩放为模型输入 张量	512x512, ImageNet 归一化
3. 模型推理	SegformerForSemanticSegmentation 前向传 播	eval 模式, inference_mode
4. 上采样	双线性插值恢复至原始分辨率	bilinear, align_corners=False
5. 概率估计	Softmax 计算各类别概率	19 类 logits
6. 类别选择	Argmax 确定每像素类别	road_class_ids = [0]
7. 掩码生成	道路类别像素置 True, 生成二值掩码	可选 confidence_threshold
8. 边界提取	四邻域腐蚀计算内边界	boundary_width = 2
9. 结构化输出	封装为 DrivableAreaPrediction 对象	mask/boundary/confidence/class_map/latency/device

系统采用本地权重加载策略(Local Model Loading), 模型从固定的本地目录加载, 使用 local_files_only=True 参数确保运行时不依赖网络下载。模型 revision(21b3847fae21ddee674abd31129307b6a1235bd9) 和权重 SHA-256(027ed78d...) 在配置中固定, 下载脚本在可用前验证模型结构、道路标签、文件大小和哈希值。这种设计可称为「离线可复现推理架构」(Offline-capable Reproducible Inference), 对于车载系统的稳定部署尤为重要。

分割模块暴露的并非原始框架张量, 而是 DrivableAreaPrediction 冻结数据类(frozen dataclass), 包含 mask(二值道路掩码)、boundary(内边界)、confidence(道路置信度图)、class_map(完整类别图)、latency_ms(推理延迟) 和 device(执行设备) 六个字段。所有像素数组在返回前统一转换为 NumPy 格式, 不泄漏 PyTorch 张量。这一抽象将模型实现与下游图像和视频处理模块解耦, 使未来的目标检测和风险评估模块可以直接消费分割结果, 无需依赖底层 PyTorch API。

2.4 目标检测模块

目标检测模块采用 YOLOv11s-P2 架构, 由项目组团队提交。该模型在 BDD100K 数据集上进行训练, 支持 15 类道路目标的检测, 包括车辆类(car、bus、truck、motor)、人员类(person、rider)、交通设施类(traffic light、traffic sign) 以及障碍物类(cone、lost_tire、obstacle、animal)。

YOLOv11s-P2 相比标准 YOLOv11s 增加了 P2 检测头(高分辨率小目标检测层), 通过在 backbone 中添加额外的上采样和 C3k2 模块, 使模型能够更好地检测远距离小目标障碍物。模型配置采用自定义 YAML 定义 backbone 和 head 结构, 支持四个尺度的检测输出(P2/P3/P4/P5), Detect 头融合多尺度特征进行最终预测。

团队提交区中已包含 SegFormer + YOLO 的融合推理原型(fusion_predict.py), 该原型演示了将

SegFormer 道路分割结果与 YOLO 车辆检测结果进行融合的初步方案:SegFormer 输出道路掩码, YOLO 输出车辆检测框, 融合模块将车辆区域从道路掩码中剔除, 得到更精确的可行驶区域。这一原型验证了互补感知设计的可行性, 但尚未集成到主工程的结构化接口中。

2.5 空间风险分析

空间风险分析是连接感知层和预警层的关键环节, 当前仍处于规划阶段。该模块的设计目标是将道路掩码、障碍物检测、距离估计和自车状态进行综合分析, 输出可解释的风险等级。系统配置中已预定义了四维状态模型:可用性、障碍物类型、空间关系和风险等级。

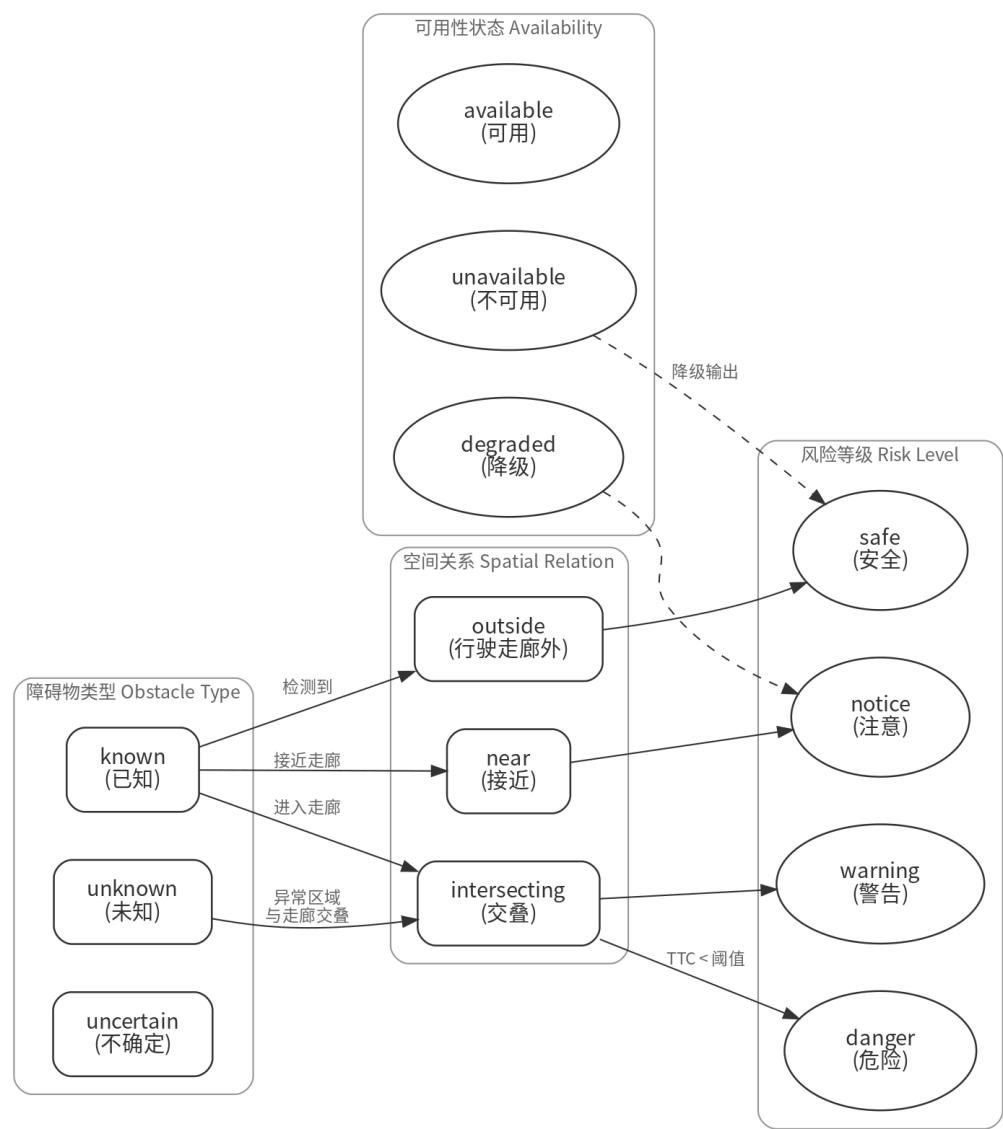


图 4: 风险状态机设计 - 从障碍物检测到风险等级的映射逻辑

风险状态机的设计逻辑如下:当已知障碍物位于行驶走廊外(outside)时, 风险等级为 safe;当障碍物接近走廊(near)时, 等级提升为 notice;当障碍物与走廊交叠(intersecting)时, 等级进一步升至 warning 或 danger, 具体取决于碰撞时间(TTC)是否低于阈值。未知障碍物一旦与走廊交叠, 同样触发高级别风险。当传感器或模型可用性降级时, 系统输出 notice 状态提醒驾驶员注意; 完全不可用时输出 safe 降级状态, 避免误报。

需要指出的是, Warning 和 Danger 的具体阈值、目标 GPU 延迟预算以及摄像头安装参数等仍为待 确认事项(在 configs/system.yaml 中保持 null), 需由产品、硬件、安全和架构方向确认后 方可进 入实现阶段。

2.6 数据流与接口设计

系统采用配置驱动(Configuration-driven)的架构设计, 所有可变参数集中在 YAML 配置文件中, 核心推理逻辑不硬编码路径、阈值或类别。配置体系分为四个层级: 路径配置、系统配置、模型配 置和实验配置, 分别由 paths.yaml、system.yaml、segformer_cityscapes.yaml 和 drivable_area_v1.yaml 管理。

配置体系结构

paths.yaml	项目根目录、源码包、数据、模型、输出 路径	路径集中管理, 禁止核心 逻辑 硬编码绝对路径
system.yaml	模块状态、状态枚举、待确认决策项	未确认参数保持 null, 禁止 硬 编码
segformer_cityscape s.y aml	模型ID、revision、权重哈希、运行设备、 道路类别、可视化参数	模型版本可追溯, 配置即文档
drivable_area_v1.yaml	实验ID、数据划分、冻结模型、输入尺寸、 指标定义、随机种子	冻结后不可原地修改, 变 更需 新建版本

推理模块支持 auto、cpu 和 cuda 三种设备模式。当配置为 auto 时, 系统在运行时检测 CUDA 可 用 性并自动选择; 当配置为 cuda 但 CUDA 不可用时, 抛出明确错误而非静默回退。CPU 模式下 可通 过 torch_threads 参数控制线程数。这一设计使同一套推理代码可以在不同计算环境下运行 : 开发阶 段使用笔记本 CPU, 测试阶段使用 NVIDIA GPU(当前基线为 RTX 4070 Laptop GPU), 最终部署阶 段可迁移至嵌入式 GPU 平台。

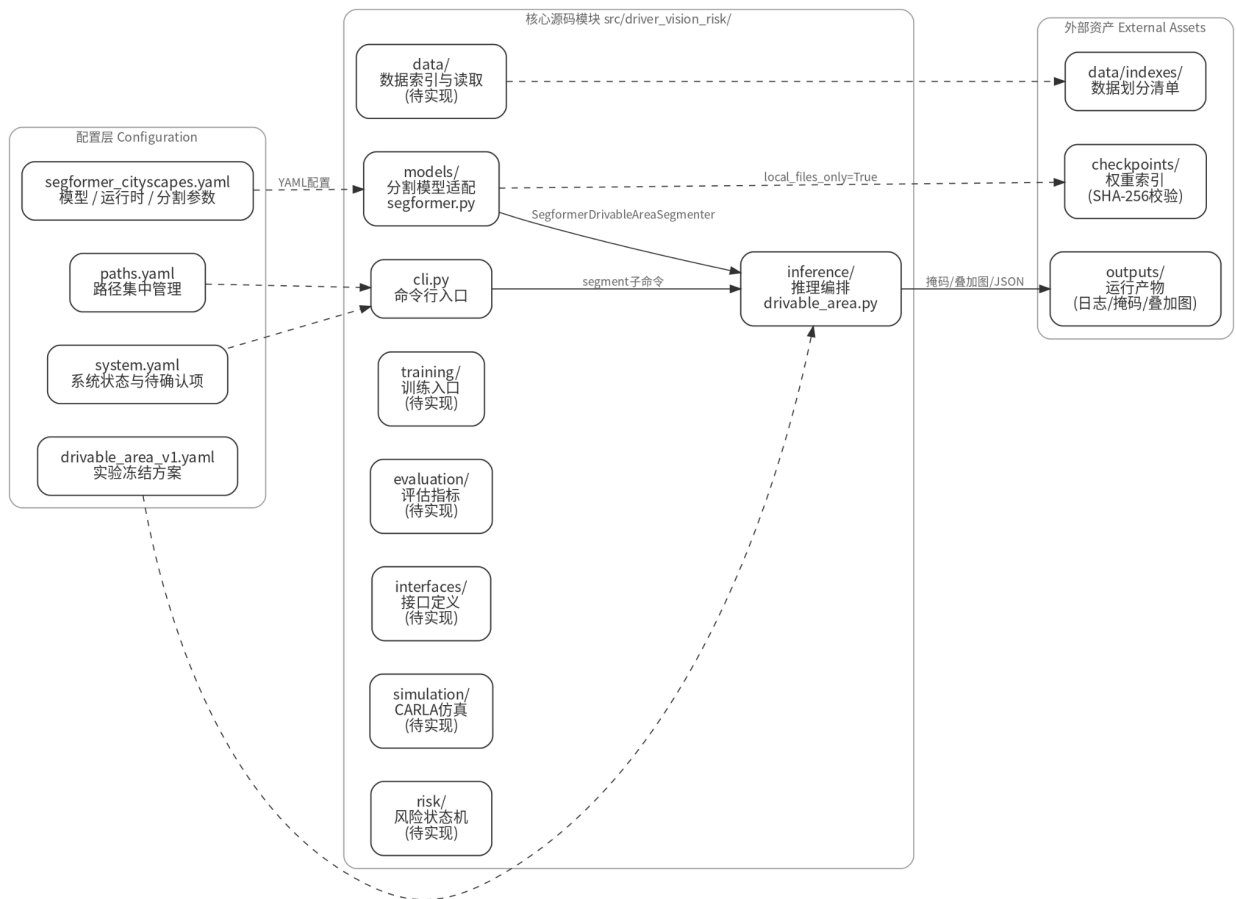


图 5: 软件模块架构 - 配置层、核心源码模块与外部资产的依赖关系

每次推理运行生成结构化的 `result.json` 元数据文件，记录输入文件路径、SHA-256 哈希、模型 ID、revision、权重哈希、运行时版本 (Python/PyTorch/Transformers)、推理设备、延迟、道路覆盖率 and 平均置信度，以及所有输出文件的 SHA-256。视频模式记录聚合指标 (帧数、平均延迟、平均覆盖率) 而非逐帧数据，避免长视频产生过大的 JSON。这种端到端的哈希链确保了从输入到输出的完整可追溯性。

2.7 架构设计决策

系统的架构设计围绕以下五个核心决策展开，这些决策共同构成了系统的架构骨架：

决策一：模块化架构 (Modular Architecture)

源码按职责拆分为 `data`、`models`、`training`、`inference`、`evaluation`、`interfaces`、`simulation` 和 `risk` 八个模块。每个模块职责独立，通过明确的接口边界交互。当前 `models` 和 `inference` 模块已实现核心功能，其余模块以 `__init__.py` 声明接口意图，等待后续开发。这种设计避免了「大泥球」式架构，使不同团队成员可以并行开发不同模块。

决策二：互补感知 (Complementary Perception)

SegFormer 提供像素级道路理解，YOLO 提供目标级障碍物检测。两个模型并非简单叠加，而是从不同维度描述同一场景：一个回答「哪里可以走」，另一个回答「路上有什么」。这种互补设计

使系 统能够同时处理已知障碍物(YOLO 可检测) 和未知异常(道路区域内 YOLO 未检测到的物体)两类风险。

决策三:配置驱动(Configuration-driven)

模型路径、运行设备、道路类别 ID、置信度阈值和可视化参数均通过 YAML 配置文件管理, 可在不 修改核心推理代码的前提下进行调整。待确认的参数(如风险阈值、目标 GPU、延迟预算)在配置 中保持 null, 禁止在核心逻辑中硬编码, 确保未经验证的参数不会进入生产代码。

决策四:硬件无关推理(Hardware-independent Inference)

通过 auto/cpu/cuda 设备抽象, 推理代码与具体硬件解耦。同一套代码可在 CPU(开发调试)、 RTX 4070(当前 GPU 基线) 和未来嵌入式 GPU 上运行。GPU 模式下通过 CUDA 同步确保延 迟统计 准确性, CPU 模式下支持线程数配置。这一设计降低了从开发到部署的迁移成本。

决策五:结构化接口(Structured Interfaces)

分割模块通过 DrivableAreaPrediction 冻结数据类暴露结构化输出, 而非返回原始 PyTorch 张 量。该对象包含 mask、boundary、confidence、class_map、latency_ms 和 device 六个字段, 所有 像 素数组已转换为 NumPy 格式。这一抽象使未来的目标检测模块(输出 object/bbox/confidence/class) 和风险评估模块可以直接消费分割结果, 实现渐进式系统集成。

2.8 当前实现与未来扩展

当前实现代表的是第一阶段感知子系统, 而非完整的端到端风险预警系统。以下表格列出了 各模块 的实现状态和集成路径:

模块	当前状态	集成路径
models/segformer.py	已实现 - SegFormer-B0 推理封装	已完成
inference/drivable_area.py	已实现 - 图像/视频推理入口	已完成
cli.py	已实现 - segment 子命令	已完成
YOLO 目标检测	团队提交 - YOLOv11s-P2 训练与融 合原型	需重构为结构化模块并集成
data/ 数据管理	仅声明 - 数据索引格式已定义	实现数据加载与质量检查
training/ 训练	仅声明 - 当前使用冻结预训练权重	实现训练入口与实验追踪
evaluation/ 评估	仅声明 - 指标定义已冻结	实现指标计算与报告生成

interfaces/ 接口	仅声明 - 接口意图已定义	实现图像/自车/障碍物/风险接口
simulation/ 仿真	仅声明 - CARLA 部署要求已定义	实现 CARLA 连通与场景采集
risk/ 风险分析	仅声明 - 状态机设计已规划	实现行驶走廊冲突与风险状态机

DrivableAreaPrediction 结构化输出接口为后续模块的接入奠定了基础。SegFormer 输出的 mask、boundary 和 confidence 可以直接作为空间推理模块的输入, 与 YOLO 输出的边界框进行交叠分析 ;同时, 道路掩码中未被已知目标覆盖的异常区域可作为未知障碍物发现的线索。这种渐进式集成 路径使系统可以在不完全重构的前提下逐步闭环。

3. 讨论 Discussion

3.1 架构优势分析

系统通过 DrivableAreaPrediction 结构化接口将模型推理与下游处理彻底解耦。分割模块返回的是 NumPy 数组而非 PyTorch 张量, 这意味着下游模块(如图像叠加、视频处理、未来的风险评估)无 需安装或理解 PyTorch 即可工作。配置驱动的参数管理进一步降低了模块间的耦合度, 模型路径、设备选择和可视化参数的变更不需要修改任何代码。

系统在可复现性方面做了充分设计:模型 revision 和权重 SHA-256 固定在配置中, 下载脚本在可用 前验证文件完整性;实验方案冻结后不可原地修改, 变更需创建新版本;每次推理运行生成包含输 入哈希、模型版本、运行时版本和输出哈希的 result.json。从数据到代码到权重的完整版本控制使 任何实验结果都可被独立验证和复现。

系统在数据集选择和模型使用上展现了严格的许可合规意识。数据集调研阶段对每个候选数 据集的 许可范围、再分发限制和商业使用条件进行了逐项审查;SegFormer 模型的 NVIDIA Source Code License 明确标注为「仅限非商业研究和评估」;每个数据集和模型的许可快照在 metadata/ 目录中 存档, 记录审查日期和用途边界。这种做法避免了项目后期因许可问题而被迫 返工的风险。

系统文档在多个层面保持了诚实的边界声明:RTX 4070 GPU 推理成功不等于满足最终目标硬 件实时 性要求;语义道路分割不等于障碍物检测(犬只被排除不等于所有未知障碍物都会被排 除); argmax 语义标签不包含安全阈值, 风险阈值必须在后续实验中校准;待确认参数保持 null 而非编造 数值。这种做法使系统状态对评审者和后续开发者保持透明。

3.2 当前架构局限

尽管架构设计具有上述优势，当前系统仍存在若干需要正视的架构局限。

当前系统仅完成了 SegFormer 可行驶区域分割环节，从分割到风险预警的完整链路尚未闭环。目标检测模块虽有团队提交代码，但未集成到主工程；空间推理、异常发现、距离估计和风险评估模块仅有接口声明而无实现。这意味着系统当前无法输出任何风险等级判段 DrivableAreaPrediction 的结构化设计虽然为后续集成预留了接口，但其实际有效性尚未得到验证。

SegFormer-B0 模型仅在 Cityscapes 数据集上完成微调，而实际评测使用的是 Lost and Found 和 RoadObstacle21 数据集。项目文档已明确指出存在域偏移 (domain shift) 问题：模型在 RoadObstacle21、乡村道路、极端天气和夜间图像上的表现可能显著下降。当前架构未包含域适应 或在线学习机制，模型一旦部署后无法根据新场景自动调整。

团队提交区中的 YOLO 训练代码和融合推理原型在功能上验证了设计可行性，但在工程质量上与主工程标准存在差距：参数硬编码 (如 DEVICE、权重路径直接写在代码中)、缺少配置文件管理、缺少哈希校验和元数据记录、未遵循主工程的模块拆分约定。将这些原型集成到主工程需要重构为符合配置驱动和结构化接口标准的模块。

当前 RTX 4070 GPU 上的单图推理延迟为 418.6 ms (包含预处理到类别图计算)，对应约 2.4 FPS，远未达到车载实时性要求。虽然视频入口已实现逐帧处理，但延迟统计不包含文件读取和可视化输出。目标 GPU 型号和延迟预算仍为待确认事项，当前性能数据仅作为本地基线，不代表最终部署平台的实时性能力。

3.3 后续扩展方向

基于当前架构分析和已知局限，系统的后续扩展可沿以下方向推进：

感知层集成与融合

首要任务是将 YOLOv11s-P2 目标检测模块重构为符合主工程标准的结构化模块，定义与 DrivableAreaPrediction 平行的检测结果数据类 (包含 bbox、class、confidence 字段)，并实现 SegFormer + YOLO 的并行推理编排。融合推理原型中「从道路掩码中剔除车辆区域」的策略可作为初始融合方案，但需要进一步扩展为更完整的空间冲突分析：不仅剔除已知车辆，还需判断所有检测目标与行驶走廊的空间关系。

风险状态机实现

基于系统配置中已预定义的四维状态模型 (可用性、障碍物类型、空间关系、风险等级)，实现可解释的风险状态机。状态机应支持配置化的阈值参数 (Warning/Danger 阈值、TTC 阈值)，在传感器降级时输出明确的降级状态。风险状态机的输出应同时包含风险等级和主要原因 (如「前方道路存在已知障碍物交叠」或「道路区域检测到未知异常」)，为驾驶员提供可理解的风险提示。

CARLA 仿真集成

项目文档已定义 CARLA 部署要求(同步模式、固定时间步长、离屏渲染)和 Ground Truth 导出规范(frame_id、仿真时间戳、自车位姿、障碍物 Actor ID、3D 包围框、距离、车道占用、风险真值)。后续应实现 simulation/ 模块的 CARLA 连通、场景管理和数据采集功能,利用仿真环境生成结构化的训练和测试数据,特别是未知异常场景的生成。仿真数据的 Ground Truth 可用于验证风险状态机的判断准确性。

实时性优化

当前 418.6 ms 的单图延迟需要通过多种手段优化:模型量化(FP16/INT8)、ONNX Runtime 或 TensorRT 推理加速、输入分辨率动态调整、多模型流水线并行等。环境基线文档已标注 ONNX Runtime 和 TensorRT 为未安装状态,后续应在目标 GPU 确认后推理框架选型和性能基准测试。延迟优化的目标应由产品和安全方向确认后制定,当前不虚构实时性门槛。

评估体系完善

V1 实验方案已冻结三项主要指标(Binary mIoU、Road IoU、Boundary F-score)和诊断指标(Road Precision/Recall/Dice、覆盖率、平均置信度)的定义,但 evaluation/ 模块尚未实现。后续应实现按数据集分别累计混淆矩阵的评估流程,并按照报告要求整理失败案例(障碍物被预测为道路、道路被预测为非道路、边界偏移、域偏移)。数值验收线必须由产品和目标硬件负责人确认后进入新版本,不编造需求文档尚未给出的门槛。